

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN HỌC PHẦN KỸ THUẬT PHẦN MỀM ỨNG DỤNG

Đề tài: Xây dựng hệ thống Custom Keyboard Builder

GVHD:

Mã lớp:

**Nhóm sinh viên
thực hiện:**

1.

MSSV:

2.

MSSV:

3.

MSSV:

4.

MSSV:

5.

MSSV:

Hà Nội, 7/2026

MỤC LỤC

<u>MỞ ĐẦU</u>	6
1. Giới thiệu và lý do chọn đề tài	6
2. Mục đích và nhiệm vụ của đề tài	6
3. Phân công nhiệm vụ của nhóm	6
4. Nội dung sơ lược của báo cáo	7
<u>CHƯƠNG 1. TỔNG QUAN HỆ THỐNG CUSTOM KEYBOARD BUILDER</u>	8
1.1. Bối cảnh và vấn đề cần giải quyết	8
1.2. Yêu cầu về dữ liệu	8
1.3. Mục tiêu và phạm vi hệ thống	9
1.4. Yêu cầu chức năng	9
1.5. Yêu cầu phi chức năng	10
1.6. Kết luận chương	10
<u>CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG</u>	12
2.1. Công nghệ và môi trường triển khai	12
2.2. Kiến trúc phần mềm	12
2.3. Use Case Diagram	14
2.4. Activity Diagram	14
2.5. Sequence Diagram	15
2.6. Entity-Relationship Diagram (ERD)	16
2.6.1. Sơ đồ ERD tổng quát	17
2.6.2. Sơ đồ ERD chi tiết	18
2.7. Đối chiếu diagram với thành phần triển khai	18
2.8. Kết luận chương	19
<u>CHƯƠNG 3. TRIỂN KHAI VÀ KIỂM THỬ</u>	20
3.1. Triển khai cơ sở dữ liệu SQL Server	20
3.1.1. Nhóm tài khoản và phân quyền	20
3.1.2. Nhóm danh mục linh kiện	21
3.1.3. Nhóm build và request	23
3.1.4. Nhóm thiết bị và kiểm tra QC	24
3.1.5. Nhóm audit và chat	25
3.2. Triển khai giao diện tiêu biểu	26
3.3. Kiểm thử và xác minh	32
3.4. Đánh giá mức độ hoàn thành	33
3.5. Kết luận chương	33
<u>KẾT LUẬN</u>	34
1. Mức độ hoàn thành nhiệm vụ	34
2. Mức độ phù hợp với mục đích đề tài	34
3. Hạn chế còn tồn tại	34
4. Hướng phát triển	34

DANH MỤC HÌNH VẼ

<u>Hình 2.1. Sơ đồ Use Case tổng hợp của hệ thống Custom Keyboard Builder</u>	14
<u>Hình 2.2. Sơ đồ Activity tổng hợp của các luồng nghiệp vụ chính</u>	15
<u>Hình 2.3. Sơ đồ Sequence tổng hợp của các luồng tương tác chính</u>	16
<u>Hình 2.4. Sơ đồ ERD tổng quát của hệ thống Custom Keyboard Builder</u>	17
<u>Hình 2.5. Sơ đồ ERD chi tiết gồm 21 bảng và 33 khóa ngoại</u>	18
<u>Hình 3.1. Giao diện đăng nhập của ứng dụng</u>	27
<u>Hình 3.2. Giao diện đăng ký tài khoản Buyer</u>	27
<u>Hình 3.3. Giao diện Buyer Dashboard và khu vực cấu hình build</u>	29
<u>Hình 3.4. Giao diện Seller Dashboard và xử lý yêu cầu lắp ráp</u>	29
<u>Hình 3.5. Giao diện Admin Dashboard và quản trị hệ thống</u>	30
<u>Hình 3.6. Giao diện trao đổi tin nhắn theo vai trò</u>	30
<u>Hình 3.7. Giao diện hồ sơ và menu người dùng</u>	31

DANH MỤC BẢNG BIỂU

<u>Bảng 0.1. Phân công nhiệm vụ nhóm</u>	6
<u>Bảng 1.1. Yêu cầu về dữ liệu</u>	8
<u>Bảng 1.2. Tác nhân và mục tiêu sử dụng hệ thống</u>	9
<u>Bảng 1.3. Yêu cầu chức năng chính</u>	9
<u>Bảng 1.4. Yêu cầu phi chức năng</u>	10
<u>Bảng 1.5. Phạm vi thực hiện và nội dung ngoài phạm vi</u>	10
<u>Bảng 2.1. Công nghệ và thư viện sử dụng</u>	12
<u>Bảng 2.2. Các lớp trong kiến trúc phần mềm</u>	13
<u>Bảng 2.3. Nhóm bảng trong cơ sở dữ liệu</u>	16
<u>Bảng 2.4. Đối chiếu bốn loại diagram với artifact nguồn</u>	18
<u>Bảng 3.1. Thứ tự triển khai 21 bảng theo nhóm phụ thuộc</u>	20
<u>Bảng 3.2. Đối chiếu giao diện với mã nguồn</u>	26
<u>Bảng 3.3. Kết quả build và xác minh tại thời điểm lập báo cáo</u>	32
<u>Bảng 3.4. Đánh giá mức độ hoàn thành theo mục tiêu</u>	33

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

- CSDL: Cơ sở dữ liệu.
- DBML: Database Markup Language - ngôn ngữ mô tả mô hình cơ sở dữ liệu.
- ERD: Entity Relationship Diagram - sơ đồ thực thể quan hệ.
- FK: Foreign Key - khóa ngoại.
- GUI: Graphical User Interface - giao diện người dùng.
- MQTT: Message Queuing Telemetry Transport - giao thức truyền thông publish/subscribe.
- MVVM: Model-View-ViewModel - mẫu kiến trúc giao diện.
- PK: Primary Key - khóa chính.
- QC: Quality Control - kiểm tra chất lượng.
- SQL: Structured Query Language - ngôn ngữ truy vấn có cấu trúc.
- SSMS: SQL Server Management Studio.
- WPF: Windows Presentation Foundation.
- XAML: Extensible Application Markup Language.

MỞ ĐẦU

1. Giới thiệu và lý do chọn đề tài

Hiện nay, việc sử dụng bàn phím cơ tự xây dựng, cá nhân hóa đang ngày càng trở nên phổ biến với những người sử dụng máy tính nhiều, một bàn phím cơ tự xây dựng có thể đáp ứng những nhu cầu riêng biệt cho từng người dùng như coder, dân văn phòng, gamer hoặc người dùng phổ thông,...

Tuy nhiên, việc tự tìm hiểu cũng như mua lẻ từng linh kiện thường gây trở ngại cho người dùng khi sự đa dạng trong lĩnh vực bàn phím là rất lớn hoặc tương thích công nghệ, để đưa ra giải pháp mà hiện nay chưa có được ở các shop bán hàng trực tuyến, dự án này nhằm thỏa mãn yêu cầu của người dùng trong việc tiện ích khi mua bán bàn phím.

Dự án Custom Keyboard Builder được xây dựng nhằm số hóa quá trình xây dựng thủ công bàn phím, gửi yêu cầu chi tiết sản phẩm cho người bán, theo dõi trạng thái đơn hàng, kiểm tra lỗi bàn phím (QC), tập trung dữ liệu sử dụng SQL Server, giao diện desktop WPF hỗ trợ cho 3 người dùng là người mua (Buyer), người bán (Seller) và quản trị viên (Admin).

2. Mục đích của đề tài

Mục đích của đề tài là xây dựng ứng dụng quản lý cấu hình bàn phím tùy chỉnh theo mô hình dựa trên kit bàn phím (kit-based), hệ thống sẽ kiểm tra tính tương thích giữa các linh kiện, cập nhật giá sản phẩm snapshot, gửi yêu cầu đến người bán và hỗ trợ quản trị viên quản trị hệ thống.

3. Phân công nhiệm vụ của nhóm (để trống phần nội dung bên trong bảng thêm vào sau)

Các vị trí họ tên và mã số sinh viên được để trống để hoàn thiện theo danh sách nhóm chính thức. Cách phân chia công việc đề xuất được trình bày tại [Bảng 0.1](#).

Bảng 0.1. Phân công nhiệm vụ nhóm

Thành viên	Nhiệm vụ được phân công	Sản phẩm/tiêu chí hoàn thành
Họ tên - MSSV:	Phân tích yêu cầu; xây dựng Use Case, Sequence và Activity Diagram.	Đặc tả tác nhân, phạm vi và luồng nghiệp vụ.
Họ tên - MSSV:	Thiết kế ERD, schema SQL Server và dữ liệu seed.	ERD 21 bảng/33 FK; DDL theo thứ tự phụ thuộc.
Họ tên - MSSV:	Triển khai WPF View, ViewModel, theme và điều hướng theo vai trò.	Login và dashboard Buyer/Seller/Admin.
Họ tên - MSSV:	Triển khai service, repository, validation, MQTT và QC mô phỏng.	Luồng build/request/chat/QC hoạt động theo thiết kế.

Thành viên	Nhiệm vụ được phân công	Sản phẩm/tiêu chí hoàn thành
Họ tên - MSSV:	Kiểm thử, seed/verify, tổng hợp tài liệu và báo cáo.	Build, source gate, ma trận kiểm thử và bản Word.

4. Nội dung sơ lược của báo cáo

Báo cáo gồm 33 trang, 3 chương chính, 12 hình vẽ, 13 bảng và 25 khối mã nguồn:

- Chương 1 Tổng quan hệ thống: trình bày bối cảnh, mục tiêu, phạm vi và yêu cầu của hệ thống.
- Chương 2 Phân tích thiết kế hệ thống: trình bày công nghệ, kiến trúc và năm sơ đồ Use Case, Activity, Sequence, ERD tổng quát và ERD chi tiết.
- Chương 3 Triển khai và kiểm thử: trình bày 21 bảng SQL, giao diện, kiểm thử và xác minh

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG CUSTOM KEYBOARD BUILDER

Chương này trình bày bối cảnh hình thành, mục tiêu, phạm vi và các yêu cầu chính của hệ thống.

1.1. Bối cảnh và vấn đề cần giải quyết

Một bộ bàn phím custom yêu cầu nhiều linh kiện cũng như tính tương thích giữa chúng, gây cản trở khó khăn cho những người mới, người mua phổ thông, vì vậy dự án này nhằm hỗ trợ người mua có thể xây dựng cho mình một bộ bàn phím cá nhân hóa, đảm bảo tính hợp lệ, giúp tiết kiệm thời gian.

Mô hình hiện tại lấy keyboard kit làm nền tảng. Một bộ Kit bao gồm case, PCB, plate và các phần đi kèm khác; Buyer tiếp tục chọn switch, keycap, stabilizer, accessory và ghi chú mod. Mẫu build_items cho phép mỗi dòng hướng đến đúng một loại sản phẩm, còn snapshot pricing hiển thị giá thành của đơn hàng tại thời điểm đó.

1.2. Yêu cầu về dữ liệu

Dữ liệu của hệ thống cần được lưu tập trung trên SQL Server, sử dụng SQL Server Management Studio (SSMS)

Các nhóm dữ liệu cần quản lý được tổng hợp tại [Bảng 1.1](#).

Bảng 1.1. Yêu cầu về dữ liệu

STT	Nhóm dữ liệu	Dữ liệu cần lưu	Yêu cầu/ràng buộc
1	Tài khoản, phân quyền và Seller	Vai trò, tên đăng nhập, email, số điện thoại, mật khẩu đã băm, trạng thái tài khoản, hồ sơ Seller và đơn đăng ký Seller.	Tên đăng nhập, email và số điện thoại không được trùng. Tài khoản không hoạt động không được đăng nhập.
2	Danh mục linh kiện	Thương hiệu, layout, keyboard kit, switch, keycap, stabilizer, phụ kiện, giá và trạng thái khả dụng.	Giá không được âm; số switch yêu cầu của kit phải lớn hơn 0. Dữ liệu phải hỗ trợ lọc và kiểm tra tương thích cơ bản.
3	Cấu hình và yêu cầu lắp ráp	Build, linh kiện đã chọn, mod, số lượng, giá snapshot, Seller tiếp nhận, nội dung JSON, trạng thái và thời điểm xử lý request.	Mỗi build item chỉ tham chiếu một loại sản phẩm. Giá và cấu hình tại thời điểm gửi request phải được giữ nguyên; trạng thái chuyển đúng luồng nghiệp vụ.
4	Kiểm tra chất lượng	Thiết bị kiểm tra, phiên QC, kết quả từng phím, độ trễ, độ ồn và kết luận đạt/cảnh báo/không đạt.	Request chỉ được hoàn thành khi phiên QC gần nhất có kết quả đạt hoặc cảnh báo.
5	Đoạn hội thoại	Hội thoại, tin nhắn, người gửi, thời điểm gửi, request liên quan và lịch sử thay đổi trước/sau.	Chỉ hỗ trợ trao đổi Buyer-Seller và Admin-Seller.

Những yêu cầu này là cơ sở để xây dựng ERD tổng quát, ERD chi tiết và các bảng SQL được trình bày ở các chương sau.

1.3. Mục tiêu và phạm vi hệ thống

Ba tác nhân chính và mục tiêu sử dụng được tổng hợp tại [Bảng 1.2](#).

Bảng 1.2. Tác nhân và mục tiêu sử dụng hệ thống

Tác nhân	Mục tiêu
Buyer	Tạo/chỉnh sửa cấu hình và đặt hàng
Seller	Tiếp nhận và xử lý request được gán.
Admin	Quản trị người dùng và dữ liệu nền.

1.4. Tác nhân và yêu cầu chức năng

Các yêu cầu chức năng cốt lõi được liệt kê tại [Bảng 1.3](#).

Bảng 1.3. Yêu cầu chức năng chính

Nhóm chức năng	Yêu cầu	Vai trò
Tài khoản	Đăng ký làm Buyer, đăng nhập bằng username/email, xem hồ sơ, đăng xuất.	Guest và Buyer
Cấu hình build	Chọn kit, switch, keycap, stabilizer, accessory, mod; xem snapshot price và lưu build.	Buyer
Request	Chọn Seller, tạo JSON snapshot, gửi và theo dõi request.	Buyer
Xử lý Request	3 trạng thái Pending - Accepted - In_progress - Completed/Cancelled.	Seller
Quản trị hệ thống	Quản lý Buyer, Seller, đơn Seller, catalog	Admin
Chat	Chỉ cho phép Buyer-Seller và Admin-Seller; lưu tin nhắn trong SQL Server.	Buyer, Seller và Admin
QC	Tạo session, mô phỏng từng phím, đánh giá latency/noise và tổng hợp pass/warning/fail.	Seller và device layer

1.5. Yêu cầu phi chức năng

Các yêu cầu chất lượng và vận hành được trình bày tại [Bảng 1.4](#).

Bảng 1.4. Yêu cầu phi chức năng

Nhóm yêu cầu	Yêu cầu áp dụng
Tính hợp lệ	Switch technology/mount phù hợp kit; tổng số switch ít nhất bằng <code>required_switch_quantity</code> .
Bảo mật	Mật khẩu lưu dạng PBKDF2-SHA256 hash; user inactive không đăng nhập.
Khả dụng	Ứng dụng hoạt động theo chế độ DB-only khi MQTT broker không khả dụng.
Dễ bảo trì	Tách View, ViewModel, Service, Repository; resource theme và localization được tổ chức riêng.
Khả năng truy vết	Thao tác quản trị quan trọng được ghi <code>audit_log</code> ; request lưu payload JSON snapshot.

1.6. Kết luận chương

Chương 1 đã xác định được bài toán, yêu cầu về dữ liệu, tác nhân, mục tiêu, yêu cầu chức năng/ phi chức năng của dự án. Dựa trên cơ sở này, Chương 2 sẽ tiếp tục mô tả công nghệ, kiến trúc, mô hình dữ liệu và các sơ đồ dùng để chuyển yêu cầu thành thiết kế kỹ thuật.

CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Chương này trình bày công nghệ, kiến trúc phần mềm và năm sơ đồ Use Case - Activity - Sequence - ERD tổng quát - ERD chi tiết.

2.1. Công nghệ và môi trường triển khai

Thành phần công nghệ được tổng hợp tại [Bảng 2.1](#).

Bảng 2.1. Công nghệ và thư viện sử dụng

Thành phần	Công nghệ/phần bản	Vai trò
Ứng dụng desktop	WPF trên .NET 10.0 Windows	Xây dựng Login, Buyer/Seller/Admin Dashboard, Chat và User Menu.
Cơ sở dữ liệu	Microsoft SQL Server Managment	Lưu tài khoản, catalog, build, request, QC, chat và audit.
Truy cập dữ liệu	Microsoft.Data.SqlClient 6.1.1; ADO.NET	Thực hiện câu lệnh SQL có tham số trong lớp repository.
Biểu đồ	LiveChartsCore.SkiaSharpView.WPF 2.0.4	Hiển thị KPI và analytics tại dashboard.
Realtime tùy chọn	MQTTnet 4.3.7; broker mặc định localhost:1883	Phát thông báo request/status và telemetry theo cơ chế best-effort.
Bảo mật	PBKDF2-SHA256	Password Hashing trước khi lưu mật khẩu trong database.
Đa ngôn ngữ	Resource/localization nội bộ	Chuyển đổi giao diện Việt-Anh.

Hiện tại dự án chưa triển khai sử dụng thực tế tầng device để gửi tín hiệu QC. Trạm QC được mô phỏng bằng DeviceSimulator; MQTT chỉ là kênh thông báo tùy chọn, còn SQL Server giữ vai trò là nguồn dữ liệu chính.

2.2. Kiến trúc phần mềm

Để hệ thống dễ bảo trì và mở rộng, ứng dụng được thiết kế tuân ngặt theo kiến trúc MVVM, giúp phân tách rõ ràng giữa giao diện (View), cấu trúc dữ liệu (Model) và logic điều khiển (ViewModel). Ở tầng dữ liệu, các lớp Repository được xây dựng để chuyên biệt hóa việc tương tác với SQL Server, nhường phần xử lý các quy tắc nghiệp vụ lỗi cho các lớp Service. Toàn bộ luồng khởi tạo được tập trung tại MainWindow – đóng vai trò như một trung tâm kết nối (Composition Root). Tại đây, các thành phần cốt lõi như Repository, Service, module MQTT, bộ mô phỏng thiết bị và MainShellViewModel được khởi tạo và tiêm (inject) vào nhau.

Về luồng người dùng, ngay khi đăng nhập thành công, MainShellViewModel sẽ đứng ra đánh giá phân quyền, từ đó linh hoạt điều hướng giao diện (Dashboard) tương ứng cho Buyer, Seller hoặc Admin. Trong khi SQL Server đóng vai trò lưu trữ dữ liệu, hệ

thống tích hợp thêm giao thức MQTT để giải quyết bài toán đẩy thông báo và dữ liệu QC theo thời gian gần thực (near real-time). Trách nhiệm cụ thể của từng lớp được trình bày chi tiết tại [Bảng 2.2](#).

Bảng 2.2. Các lớp trong kiến trúc phần mềm

Lớp	Thành phần	Trách nhiệm
View	Views/*.xaml	Khai báo bố cục, binding, command, DataGrid, chart và trạng thái hiển thị.
ViewModel	ViewModels/*ViewModel.cs	Quản lý state giao diện, command bất đồng bộ và điều phối service.
Service	Services/*.cs	Thực thi quy tắc nghiệp vụ, validation, state machine và phân quyền.
Repository	Repositories/SqlServer/*.cs	Đọc/ghi SQL Server bằng Microsoft.Data.SqlClient và ánh xạ model.
Data/Realtime	Data/SqlServer; Realtime	Tạo connection, kiểm tra schema và publish/subscribe MQTT best-effort.

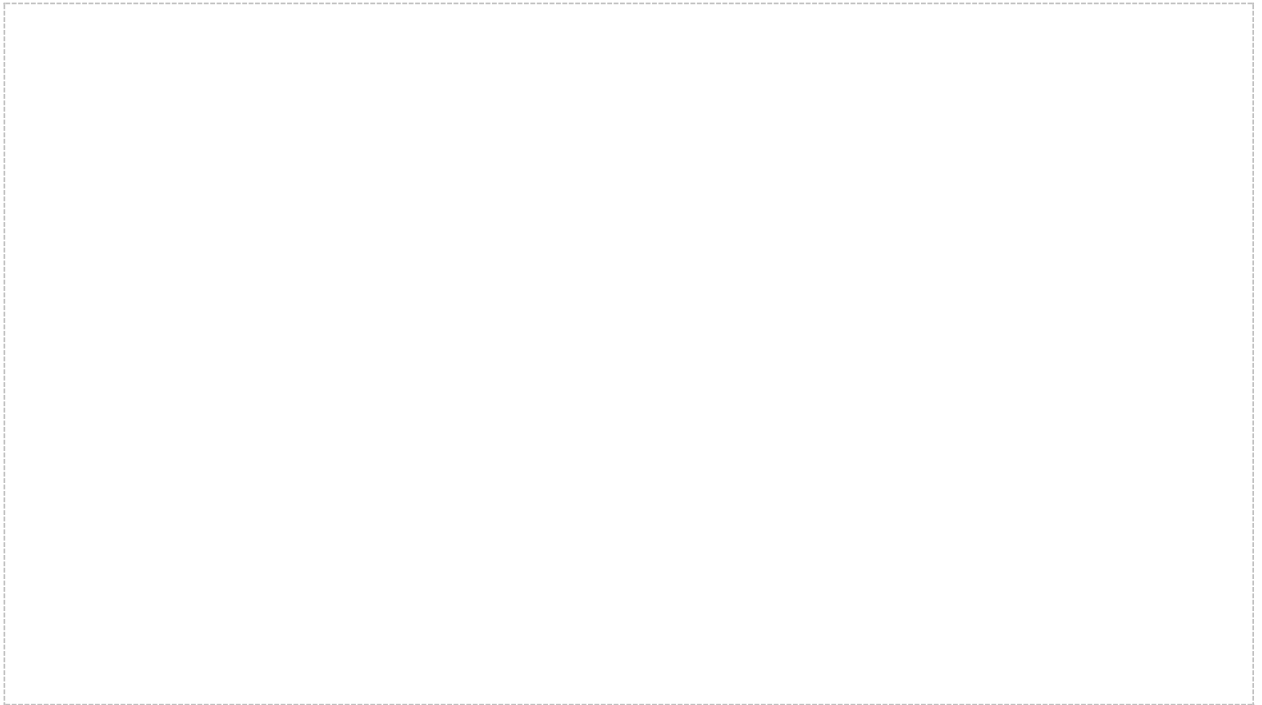
Note: Bổ sung thêm lên trên cùng của bảng: Lớp: Model; Thành phần: Models/**/**.cs; Trách nhiệm: Biểu diễn dữ liệu và các đối tượng nghiệp vụ của hệ thống; được Repository, Service và ViewModel sử dụng.

2.3. Use Case Diagram

Use Case Diagram xác định ranh giới chức năng theo ba tác nhân Buyer, Seller và Admin. Nguồn thiết kế chỉ có UC-01 Buyer, UC-02 Seller và UC-03 Admin; Device/QC là thành phần hỗ trợ luồng Seller, không tạo thành UC-04 độc lập.

Sơ đồ cần tổng hợp ba tác nhân Buyer, Seller và Admin; phạm vi QC được đặt trong luồng Seller, không tách thành tác nhân độc lập. Vị trí chèn thủ công được đánh dấu tại [Hình 2.1](#).

*Nguồn diagram: Documents_Refactor/Custom_Keyboard_Use_Cases_Refactor.md;
KTPMUD_diagram/16-18_UseCase_*.drawio*



Hình 2.1. Sơ đồ Use Case tổng hợp của hệ thống Custom Keyboard Builder

2.4. Activity Diagram

Activity Diagram mô hình hóa các bước xử lý và nhánh quyết định của sáu nhóm hoạt động: tài khoản, tạo build/request, Seller xử lý request, Admin quản trị, chat và kiểm tra Device/QC.

Sơ đồ cần tổng hợp các nhánh hoạt động về tài khoản, Buyer, Seller, Admin, chat và kiểm tra Device/QC. Vị trí chèn thủ công được đánh dấu tại [Hình 2.2](#).

Nguồn diagram: Documents_Refactor/Custom_Keyboard_Activity_Diagrams_Refactor.md



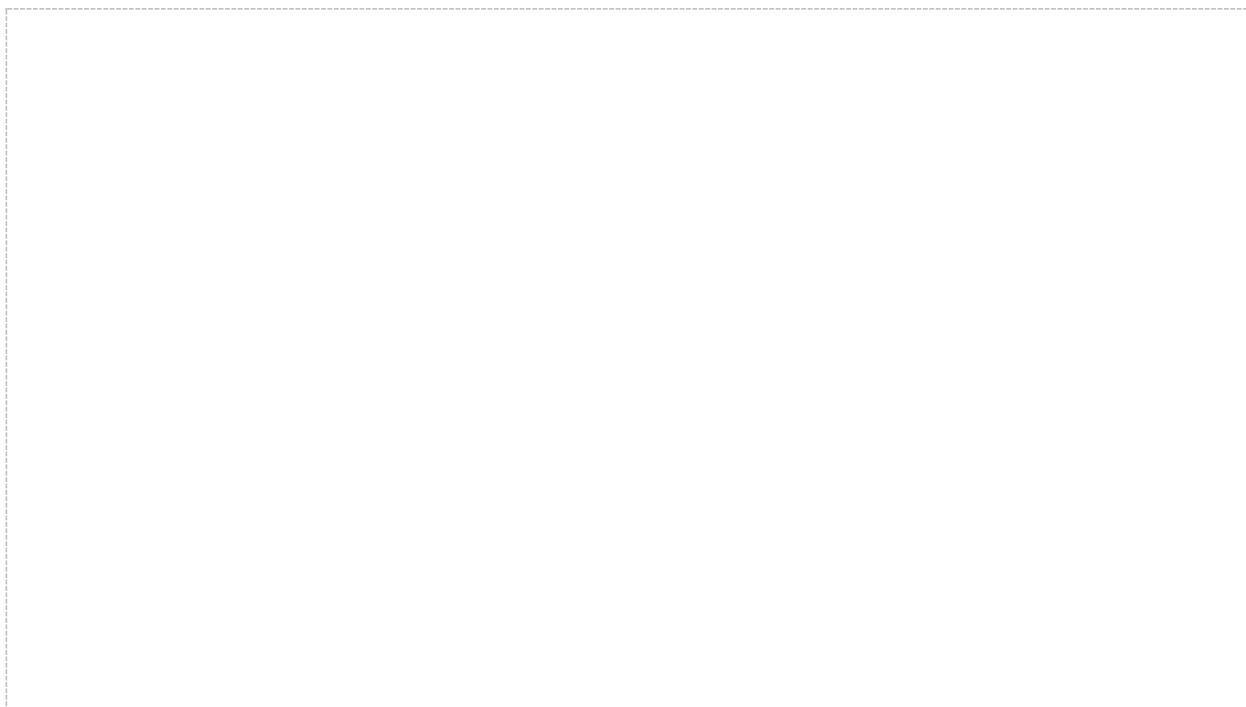
Hình 2.2. Sơ đồ Activity tổng hợp của các luồng nghiệp vụ chính

2.5. Sequence Diagram

Sequence Diagram mô tả thứ tự thông điệp giữa người dùng, View/ViewModel, Service, Repository, SQL Server và thành phần realtime. Sáu luồng lỗi bao phủ tài khoản, build request, Seller/QC, quản trị, duyệt đơn Seller và chat.

Sơ đồ cần tổng hợp trình tự giữa View/ViewModel, Service, Repository, SQL Server và thành phần realtime trong sáu luồng lỗi. Vị trí chèn thủ công được đánh dấu tại [Hình 2.3](#).

Nguồn diagram: Documents_Refactor/Custom_Keyboard_Sequence_Diagrams_6_Core.md



Hình 2.3. Sơ đồ Sequence tổng hợp của các luồng tương tác chính

2.6. Entity-Relationship Diagram (ERD)

ERD chuẩn được mô tả bằng DBML trong Documents_Refactor. Mô hình gồm 21 bảng nghiệp vụ, 21 khóa chính đều tên id và 33 khóa ngoại; dbo.schema_migrations là bảng hạ tầng nên không được tính vào ERD nghiệp vụ.

Các nhóm bảng được tổng hợp tại [Bảng 2.3](#).

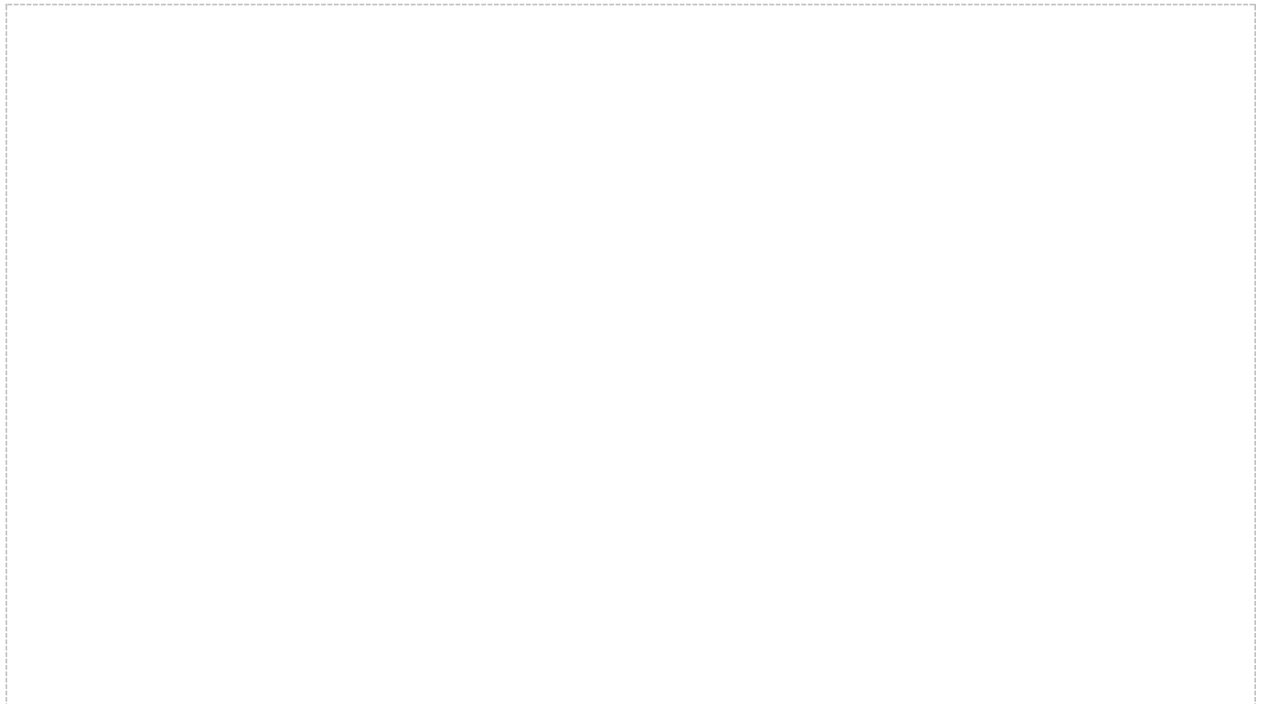
Bảng 2.3. Nhóm bảng trong cơ sở dữ liệu

Nhóm	Bảng	Mục đích
Tài khoản	roles, users, seller_profiles, seller_applications	Role, tài khoản, hồ sơ Seller và quy trình nâng cấp Buyer thành Seller.
Catalog	brands, layouts, keyboard_kits, switches, keycap_sets, stabilizers, accessories	Danh mục kit-based và thông tin tương thích cơ bản.
Build/Request	builds, build_items, build_mods, build_requests	Cấu hình, giá snapshot, mod note và yêu cầu gửi Seller.
Device/QC	devices, device_test_sessions, device_key_test_results	Trạm QC, phiên kiểm tra và kết quả từng phím.
Hỗ trợ	audit_log, chat_conversations, chat_messages	Truy vết và trao đổi theo role.

2.6.1. Sơ đồ ERD tổng quát

Sơ đồ cần thể hiện năm vùng dữ liệu: tài khoản, danh mục, build/request, Device/QC và audit/chat cùng các quan hệ chính. Vị trí chèn thủ công được đánh dấu tại [Hình 2.4](#).

Nguồn diagram: *Documents_Refactor/Custom_Keyboard_ERD_Realistic_Kit_Shop_Proposal.dbml*



Hình 2.4. Sơ đồ ERD tổng quát của hệ thống Custom Keyboard Builder

2.6.2. Sơ đồ ERD chi tiết

Sơ đồ cần hiển thị cột, kiểu dữ liệu, khóa chính id, khóa ngoại, UNIQUE, CHECK và hai filtered/unique index có ý nghĩa nghiệp vụ. Vị trí chèn thủ công được đánh dấu tại [Hình 2.5](#).

Nguồn diagram: Documents_Refactor/Custom_Keyboard_ERD_Realistic_Kit_Shop_Proposal.dbml;
Database/SqlServer/CreateSchema_Refactor.sql



Hình 2.5. Sơ đồ ERD chi tiết gồm 21 bảng và 33 khóa ngoại

2.7. Đối chiếu diagram với thành phần triển khai

Nguồn của năm sơ đồ thuộc bốn nhóm ký pháp được đối chiếu tại [Bảng 2.4](#).

Bảng 2.4. Đối chiếu bốn loại diagram với artifact nguồn

Nhóm ký pháp	Artifact nguồn	Phạm vi phản ánh
Use Case	Custom_Keyboard_Use_Cases_Refactor.md; Draw.io 16-18	Tác nhân và chức năng theo role; UC-02 bao gồm QC.
Activity	Custom_Keyboard_Activity_Diagrams_Refactor.md	Nhánh quyết định của sáu nhóm hoạt động.
Sequence	Custom_Keyboard_Sequence_Diagrams_6_Core.md	Tương tác theo thời gian của sáu luồng lỗi.
ERD	Custom_Keyboard_ERD_Realistic_Kit_Shop_Proposal.dbml	Một sơ đồ tổng quát và một sơ đồ chi tiết cho 21 bảng, 33 FK.

2.8. Kết luận chương

Chương 2 đã xác định công nghệ, kiến trúc, luồng chức năng và mô hình dữ liệu thông qua năm sơ đồ thuộc bốn nhóm ký pháp. ERD chi tiết là cơ sở trực tiếp để trình bày 21 khối DDL SQL Server, giao diện và kết quả kiểm thử tại Chương 3.

CHƯƠNG 3. TRIỂN KHAI VÀ KIỂM THỬ

Chương này trình bày DDL của từng bảng theo ERD, giao diện và mã nguồn tiêu biểu, kết quả build/xác minh cùng đánh giá mức độ hoàn thành. Tất cả đoạn code được đưa dưới dạng văn bản trong bảng hai hàng, không sử dụng ảnh chụp code.

3.1. Triển khai cơ sở dữ liệu SQL Server

CreateSchema_Refactor.sql được dùng cho môi trường clean/disposable và có hành vi xóa, tạo lại 21 bảng. Database có dữ liệu phải dùng chuỗi preflight - migration PK-to-id - postflight trên bản restore rehearsal; không được chạy clean schema trực tiếp.

Tại thời điểm audit ngày 15/07/2026, source code, DBML, schema và repository đã thống nhất khóa chính id, nhưng runtime database vẫn dùng 21 tên PK legacy. Vì vậy migration được xem là đã chuẩn bị nhưng chưa được xác nhận triển khai trên runtime.

Thứ tự phụ thuộc của các nhóm bảng được tóm tắt tại [Bảng 3.1](#).

Bảng 3.1. Thứ tự triển khai 21 bảng theo nhóm phụ thuộc

Thứ tự	Nhóm	Bảng	Số bảng
1	Tài khoản	roles, users, seller_profiles, seller_applications	4
2	Catalog	brands, layouts, keyboard_kits, switches, keycap_sets, stabilizers, accessories	7
3	Build/Request	builds, build_items, build_mods, build_requests	4
4	Device/QC	devices, device_test_sessions, device_key_test_results	3
5	Audit/Chat	audit_log, chat_conversations, chat_messages	3

3.1.1. Nhóm tài khoản và phân quyền

Mã SQL 3.1 - Tạo bảng roles

```
CREATE TABLE roles (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  role_name VARCHAR(50) NOT NULL UNIQUE,  
  permissions VARCHAR(MAX) NULL  
);
```

Mã SQL 3.2 - Tạo bảng users

```
CREATE TABLE users (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  role_id INT NOT NULL,  
  username VARCHAR(100) NOT NULL UNIQUE,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  phone VARCHAR(30) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  is_active BIT NOT NULL DEFAULT 1,  
  CONSTRAINT FK_users_roles FOREIGN KEY (role_id) REFERENCES roles(id)  
);
```

Mã SQL 3.3 - Tạo bảng seller_profiles

```
CREATE TABLE seller_profiles (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  user_id INT NOT NULL UNIQUE,  
  shop_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(30) NOT NULL,  
  address VARCHAR(500) NOT NULL,  
  is_verified BIT NOT NULL DEFAULT 0,  
  verified_at DATETIME2 NULL,  
  CONSTRAINT FK_seller_profiles_user FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Mã SQL 3.4 - Tạo bảng seller_applications

```
CREATE TABLE seller_applications (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  buyer_user_id INT NOT NULL,  
  shop_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(30) NOT NULL,  
  address VARCHAR(500) NOT NULL,  
  note VARCHAR(500) NULL,  
  status VARCHAR(50) NOT NULL DEFAULT 'Pending',  
  review_note VARCHAR(500) NULL,  
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
  reviewed_at DATETIME2 NULL,  
  reviewed_by INT NULL,  
  CONSTRAINT FK_seller_applications_buyer FOREIGN KEY (buyer_user_id) REFERENCES users(id),  
  CONSTRAINT FK_seller_applications_reviewer FOREIGN KEY (reviewed_by) REFERENCES users(id),  
  CONSTRAINT CK_seller_applications_status CHECK (status IN ('Pending', 'Approved', 'Rejected'))  
);  
  
CREATE UNIQUE INDEX UX_seller_applications_pending_buyer  
ON seller_applications(buyer_user_id)  
WHERE status = 'Pending';
```

3.1.2. Nhóm danh mục linh kiện

Mã SQL 3.5 - Tạo bảng brands

```
CREATE TABLE brands (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  brand_name VARCHAR(255) NOT NULL UNIQUE,  
  country VARCHAR(100) NULL  
);
```

Mã SQL 3.6 - Tạo bảng layouts

```
CREATE TABLE layouts (  
  id VARCHAR(50) PRIMARY KEY,  
  layout_name VARCHAR(100) NOT NULL,  
  form_factor VARCHAR(100) NOT NULL,  
  key_count INT NOT NULL  
);
```

Mã SQL 3.7 - Tạo bảng keyboard_kits

```
CREATE TABLE keyboard_kits (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    layout_id VARCHAR(50) NOT NULL,  
    kit_name VARCHAR(255) NOT NULL,  
    pcb_technology VARCHAR(50) NOT NULL,  
    switch_mount VARCHAR(100) NOT NULL,  
    required_switch_quantity INT NOT NULL,  
    included_parts VARCHAR(500) NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_keyboard_kits_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT FK_keyboard_kits_layouts FOREIGN KEY (layout_id) REFERENCES layouts(id),  
    CONSTRAINT CK_keyboard_kits_price CHECK (price_usd >= 0),  
    CONSTRAINT CK_keyboard_kits_switch_qty CHECK (required_switch_quantity > 0)  
);
```

Mã SQL 3.8 - Tạo bảng switches

```
CREATE TABLE switches (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    switch_name VARCHAR(255) NOT NULL,  
    switch_technology VARCHAR(50) NOT NULL,  
    mount_type VARCHAR(100) NOT NULL,  
    switch_type VARCHAR(100) NULL,  
    actuation_force_g INT NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_switches_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT CK_switches_price CHECK (price_usd >= 0)  
);
```

Mã SQL 3.9 - Tạo bảng keycap_sets

```
CREATE TABLE keycap_sets (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    keycap_name VARCHAR(255) NOT NULL,  
    supported_form_factor VARCHAR(255) NOT NULL,  
    profile VARCHAR(100) NULL,  
    material VARCHAR(100) NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_keycap_sets_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT CK_keycap_sets_price CHECK (price_usd >= 0)  
);
```

Mã SQL 3.10 - Tạo bảng stabilizers

```
CREATE TABLE stabilizers (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    stab_name VARCHAR(255) NOT NULL,  
    supported_layouts VARCHAR(255) NOT NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_stabilizers_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT CK_stabilizers_price CHECK (price_usd >= 0)  
);
```

Mã SQL 3.11 - Tạo bảng accessories

```
CREATE TABLE accessories (  
  id VARCHAR(50) PRIMARY KEY,  
  accessory_type VARCHAR(100) NOT NULL,  
  accessory_name VARCHAR(255) NOT NULL,  
  target_component VARCHAR(100) NULL,  
  price_usd DECIMAL(10,2) NOT NULL,  
  is_available BIT NOT NULL DEFAULT 1,  
  CONSTRAINT CK_accessories_price CHECK (price_usd >= 0)  
);
```

3.1.3. Nhóm build và request

Mã SQL 3.12 - Tạo bảng builds

```
CREATE TABLE builds (  
  id VARCHAR(50) PRIMARY KEY,  
  buyer_id INT NOT NULL,  
  kit_id VARCHAR(50) NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  notes VARCHAR(500) NULL,  
  noise_requirement VARCHAR(20) NOT NULL DEFAULT 'Normal',  
  status VARCHAR(50) NOT NULL,  
  total_cost_snapshot DECIMAL(10,2) NOT NULL,  
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
  updated_at DATETIME2 NULL,  
  CONSTRAINT FK_builds_buyer FOREIGN KEY (buyer_id) REFERENCES users(id),  
  CONSTRAINT FK_builds_kit FOREIGN KEY (kit_id) REFERENCES keyboard_kits(id),  
  CONSTRAINT CK_builds_noise_requirement CHECK (noise_requirement IN ('Normal', 'Quiet', 'Silent')),  
  CONSTRAINT CK_builds_status CHECK (status IN ('Draft', 'Saved', 'Requested', 'Archived')),  
  CONSTRAINT CK_builds_total CHECK (total_cost_snapshot >= 0)  
);
```

Mã SQL 3.13 - Tạo bảng build_items

```
CREATE TABLE build_items (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  build_id VARCHAR(50) NOT NULL,  
  switch_id VARCHAR(50) NULL,  
  keycap_id VARCHAR(50) NULL,  
  stab_id VARCHAR(50) NULL,  
  accessory_id VARCHAR(50) NULL,  
  quantity INT NOT NULL,  
  unit_price_snapshot DECIMAL(10,2) NOT NULL,  
  notes VARCHAR(500) NULL,  
  CONSTRAINT FK_build_items_build FOREIGN KEY (build_id) REFERENCES builds(id),  
  CONSTRAINT FK_build_items_switch FOREIGN KEY (switch_id) REFERENCES switches(id),  
  CONSTRAINT FK_build_items_keycap FOREIGN KEY (keycap_id) REFERENCES keycap_sets(id),  
  CONSTRAINT FK_build_items_stab FOREIGN KEY (stab_id) REFERENCES stabilizers(id),  
  CONSTRAINT FK_build_items_accessory FOREIGN KEY (accessory_id) REFERENCES accessories(id),  
  CONSTRAINT CK_build_items_quantity CHECK (quantity > 0),  
  CONSTRAINT CK_build_items_unit_price CHECK (unit_price_snapshot >= 0),  
  
  CONSTRAINT CK_build_items_exactly_one_fk CHECK (  
    (CASE WHEN switch_id IS NULL THEN 0 ELSE 1 END) +  
    (CASE WHEN keycap_id IS NULL THEN 0 ELSE 1 END) +  
    (CASE WHEN stab_id IS NULL THEN 0 ELSE 1 END) +  
    (CASE WHEN accessory_id IS NULL THEN 0 ELSE 1 END) = 1  
  )  
);
```

Mã SQL 3.14 - Tạo bảng build_mods

```
CREATE TABLE build_mods (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  build_id VARCHAR(50) NOT NULL,  
  mod_type VARCHAR(100) NOT NULL,  
  target_component VARCHAR(100) NOT NULL,  
  notes VARCHAR(500) NULL,  
  CONSTRAINT FK_build_mods_build FOREIGN KEY (build_id) REFERENCES builds(id)  
);
```

Mã SQL 3.15 - Tạo bảng build_requests

```
CREATE TABLE build_requests (
  id VARCHAR(50) PRIMARY KEY,
  build_id VARCHAR(50) NOT NULL,
  seller_user_id INT NOT NULL,
  request_payload_json NVARCHAR(MAX) NOT NULL,
  status VARCHAR(50) NOT NULL,
  note VARCHAR(500) NULL,
  requested_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  accepted_at DATETIME2 NULL,
  completed_at DATETIME2 NULL,
  updated_at DATETIME2 NULL,
  CONSTRAINT FK_build_requests_build FOREIGN KEY (build_id) REFERENCES builds(id),
  CONSTRAINT FK_build_requests_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT CK_build_requests_status CHECK (status IN ('Pending', 'Accepted', 'In_progress', 'Completed',
'Cancelled'))
);
```

3.1.4. Nhóm thiết bị và kiểm tra QC

Mã SQL 3.16 - Tạo bảng devices

```
CREATE TABLE devices (
  id VARCHAR(50) PRIMARY KEY,
  seller_user_id INT NOT NULL,
  device_name NVARCHAR(100) NOT NULL,
  device_type VARCHAR(50) NOT NULL,
  is_active BIT NOT NULL DEFAULT 1,
  last_seen_at DATETIME2 NULL,
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_devices_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT CK_devices_type CHECK (device_type IN
('QC_STATION', 'KEY_SIGNAL_TESTER', 'LATENCY_TESTER', 'NOISE_SENSOR'))
);
```

Mã SQL 3.17 - Tạo bảng device_test_sessions

```
CREATE TABLE device_test_sessions (
  id VARCHAR(50) PRIMARY KEY,
  request_id VARCHAR(50) NOT NULL,
  device_id VARCHAR(50) NOT NULL,
  seller_user_id INT NOT NULL,
  switch_technology VARCHAR(50) NOT NULL,
  noise_requirement VARCHAR(20) NOT NULL DEFAULT 'Normal',
  total_keys INT NOT NULL,
  tested_keys INT NOT NULL DEFAULT 0,
  passed_keys INT NOT NULL DEFAULT 0,
  warning_keys INT NOT NULL DEFAULT 0,
  failed_keys INT NOT NULL DEFAULT 0,
  average_latency_ms DECIMAL(8,2) NULL,
  max_latency_ms DECIMAL(8,2) NULL,
  average_noise_db DECIMAL(8,2) NULL,
  max_noise_db DECIMAL(8,2) NULL,
  status VARCHAR(20) NOT NULL,
  started_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  completed_at DATETIME2 NULL,
  CONSTRAINT FK_dts_request FOREIGN KEY (request_id) REFERENCES build_requests(id),
  CONSTRAINT FK_dts_device FOREIGN KEY (device_id) REFERENCES devices(id),
  CONSTRAINT FK_dts_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT CK_dts_noise_requirement CHECK (noise_requirement IN ('Normal', 'Quiet', 'Silent')),
  CONSTRAINT CK_dts_status CHECK (status IN ('Running', 'Passed', 'Warning', 'Failed'))
);
```

Mã SQL 3.18 - Tạo bảng device_key_test_results

```
CREATE TABLE device_key_test_results (
  id BIGINT IDENTITY(1,1) PRIMARY KEY,
  session_id VARCHAR(50) NOT NULL,
  request_id VARCHAR(50) NOT NULL,
  device_id VARCHAR(50) NOT NULL,
  key_code VARCHAR(30) NOT NULL,
  expected_key VARCHAR(30) NOT NULL,
  received_key VARCHAR(30) NULL,
  press_signal_detected BIT NOT NULL,
  latency_ms DECIMAL(8,2) NULL,
  press_event_count INT NOT NULL,
  bounce_count INT NULL,
  release_signal_detected BIT NOT NULL,
  hold_duration_ms INT NULL,
  is_stuck BIT NOT NULL,
  noise_db DECIMAL(8,2) NULL,
  switch_technology VARCHAR(50) NOT NULL,
  result VARCHAR(20) NOT NULL,
  failure_type VARCHAR(30) NULL,
  failure_reason NVARCHAR(255) NULL,
  recorded_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_dktr_session FOREIGN KEY (session_id) REFERENCES device_test_sessions(id),
  CONSTRAINT FK_dktr_request FOREIGN KEY (request_id) REFERENCES build_requests(id),
  CONSTRAINT FK_dktr_device FOREIGN KEY (device_id) REFERENCES devices(id),
  CONSTRAINT CK_dktr_result CHECK (result IN ('Pass', 'Warning', 'Fail')),
  CONSTRAINT CK_dktr_failure_type CHECK (
    failure_type IS NULL
    OR failure_type IN ('NoSignal', 'WrongKey', 'Chatter', 'StuckKey', 'HighLatency', 'TooNoisy')
  )
);

CREATE UNIQUE INDEX UX_dktr_session_key_code
ON device_key_test_results(session_id, key_code);
```

3.1.5. Nhóm audit và chat

Mã SQL 3.19 - Tạo bảng audit_log

```
CREATE TABLE audit_log (
  id INT IDENTITY(1,1) PRIMARY KEY,
  user_id INT NOT NULL,
  table_name VARCHAR(100) NOT NULL,
  record_id VARCHAR(100) NOT NULL,
  action VARCHAR(100) NOT NULL,
  old_value_json NVARCHAR(MAX) NULL,
  new_value_json NVARCHAR(MAX) NULL,
  changed_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_audit_log_user FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Mã SQL 3.20 - Tạo bảng chat_conversations

```
CREATE TABLE chat_conversations (
  id VARCHAR(50) PRIMARY KEY,
  seller_user_id INT NOT NULL,
  buyer_id INT NULL,
  admin_user_id INT NULL,
  build_request_id VARCHAR(50) NULL,
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  updated_at DATETIME2 NULL,
  CONSTRAINT FK_chat_conversations_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT FK_chat_conversations_buyer FOREIGN KEY (buyer_id) REFERENCES users(id),
  CONSTRAINT FK_chat_conversations_admin FOREIGN KEY (admin_user_id) REFERENCES users(id),
  CONSTRAINT FK_chat_conversations_request FOREIGN KEY (build_request_id) REFERENCES build_requests(id),

  CONSTRAINT CK_chat_conversations_participant CHECK (
    (CASE WHEN buyer_id IS NULL THEN 0 ELSE 1 END) +
    (CASE WHEN admin_user_id IS NULL THEN 0 ELSE 1 END) = 1
  )
);
```


Mã SQL 3.21 - Tạo bảng chat_messages

```
CREATE TABLE chat_messages (  
    id VARCHAR(50) PRIMARY KEY,  
    conversation_id VARCHAR(50) NOT NULL,  
    sender_user_id INT NOT NULL,  
    message_text NVARCHAR(MAX) NOT NULL,  
    sent_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    CONSTRAINT FK_chat_messages_conversation FOREIGN KEY (conversation_id) REFERENCES chat_conversations(id),  
    CONSTRAINT FK_chat_messages_sender FOREIGN KEY (sender_user_id) REFERENCES users(id)  
);
```

3.2. Triển khai giao diện tiêu biểu

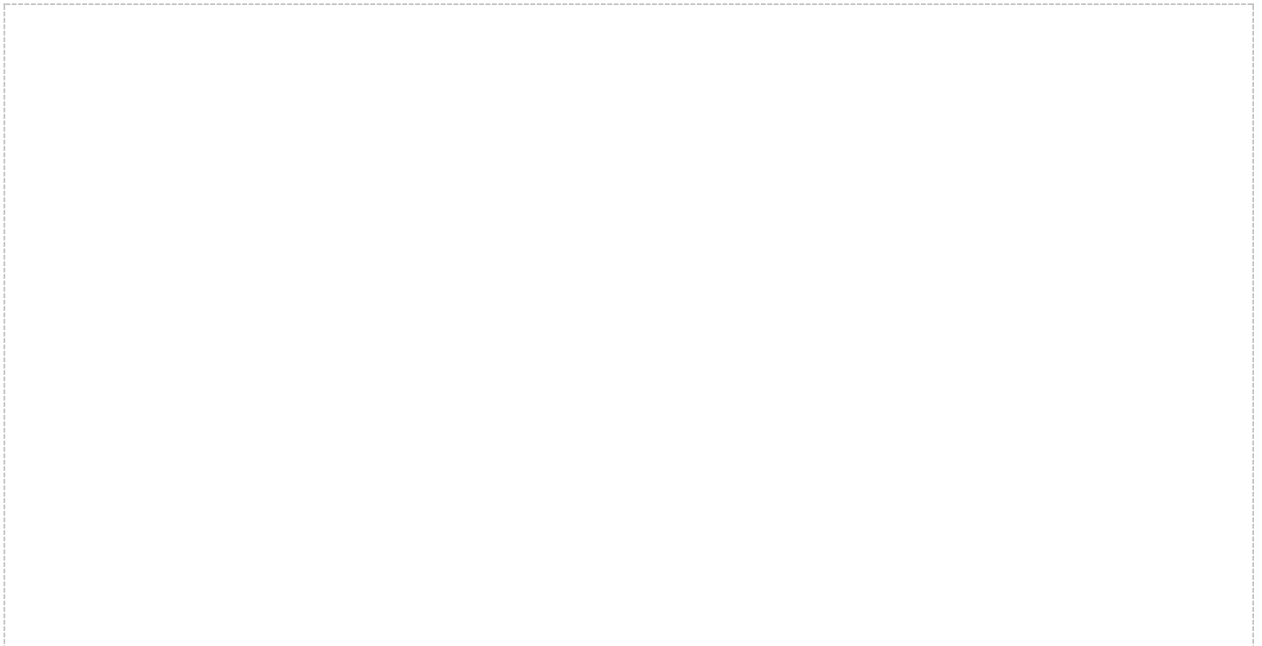
Mối liên hệ giữa giao diện và mã nguồn được tổng hợp tại [Bảng 3.2](#).

Bảng 3.2. Đối chiếu giao diện với mã nguồn

Giao diện	View	ViewModel/Service	Nội dung minh họa
Đăng nhập	Views/LoginView.xaml	LoginViewModel; AccountService	Nhập thông tin, xác thực, báo lỗi và điều hướng theo role.
Đăng ký	Views/RegisterView.xaml	RegisterViewModel; AccountService	Đăng ký Buyer và kiểm tra dữ liệu đầu vào.
Buyer Dashboard	Views/ BuyerDashboard View.xaml	BuyerDashboardViewModel; BuildService; RequestService	Cấu hình build, tính giá, validation và gửi request.
Seller Dashboard	Views/ SellerDashboard View.xaml	SellerDashboardViewModel; RequestService; DeviceService	Xử lý request, trạng thái và QC mô phỏng.
Admin Dashboard	Views/ AdminDashboard View.xaml	AdminDashboardViewModel; AdminService; StatsService	KPI, user, Seller, catalog, application và audit.
Chat	Views/ChatView.xaml	ChatViewModel; ChatService	Hội thoại Buyer-Seller và Admin-Seller có kiểm tra quyền.
User Menu	Views/ UserMenuView.xaml	MainShellViewModel	Hồ sơ, ngôn ngữ và đăng xuất.

Vùng hình dành cho ảnh chụp màn hình đăng nhập, thông báo trạng thái và điều hướng tạo tài khoản. Vị trí chèn thủ công được đánh dấu tại [Hình 3.1](#).

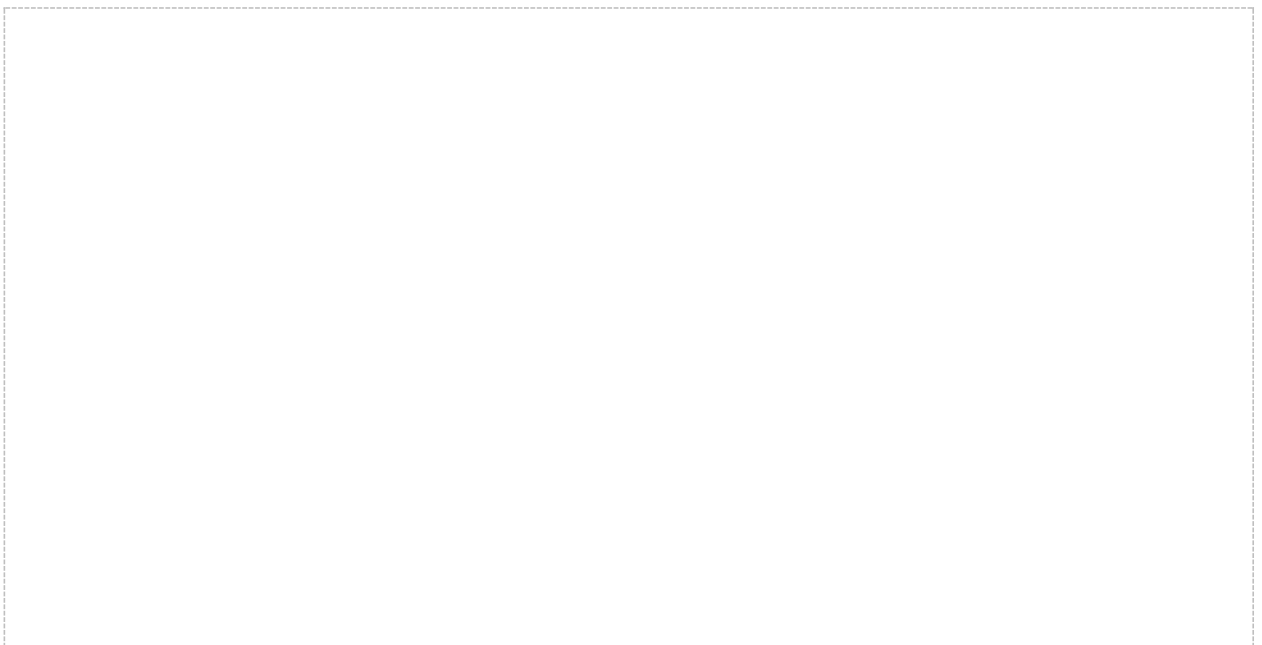
Nguồn diagram: Views/LoginView.xaml



Hình 3.1. Giao diện đăng nhập của ứng dụng

Vùng hình dành cho ảnh chụp biểu mẫu đăng ký, kiểm tra dữ liệu đầu vào và phản hồi kết quả. Vị trí chèn thủ công được đánh dấu tại [Hình 3.2](#).

Nguồn diagram: Views/RegisterView.xaml



Hình 3.2. Giao diện đăng ký tài khoản Buyer

Mã nguồn 3.22 - Form đăng nhập bằng XAML

```
<TextBlock Text="{loc:Tr Login_EmailOrUsername}"
           Style="{StaticResource FieldLabel}"/>
<TextBox AutomationProperties.AutomationId="LoginEmailOrUsernameTextBox"
          Text="{Binding EmailOrUsername, UpdateSourceTrigger=PropertyChanged}"/>

<TextBlock Text="{loc:Tr Common_Password}"
           Style="{StaticResource FieldLabel}"/>
<PasswordBox AutomationProperties.AutomationId="LoginPasswordBox"
              behaviors:PasswordBoxBinding.Attach="True"
              behaviors:PasswordBoxBinding.BoundPassword="{Binding Password, Mode=TwoWay}"/>

<TextBlock Text="{Binding ErrorMessage}"
           Foreground="{StaticResource DangerBrush}"/>
<Button Content="{loc:Tr Login_SignIn}"
        Command="{Binding LoginCommand}"
        Style="{StaticResource PrimaryButton}"/>
```

Mã nguồn 3.23 - ViewModel xử lý đăng nhập

```
private async Task LoginAsync()
{
    ErrorMessage = string.Empty;
    StatusMessage = string.Empty;

    var result = await _accountService.LoginAsync(
        EmailOrUsername,
        Password);

    if (result.Succeeded && result.User is not null)
    {
        Password = string.Empty;
        _loginSucceeded(result.User);
        return;
    }

    ErrorMessage = result.Message;
}
```

Vùng hình dành cho ảnh chụp configurator kit-based, tổng giá snapshot, validation và thao tác gửi request. Vị trí chèn thủ công được đánh dấu tại [Hình 3.3](#).

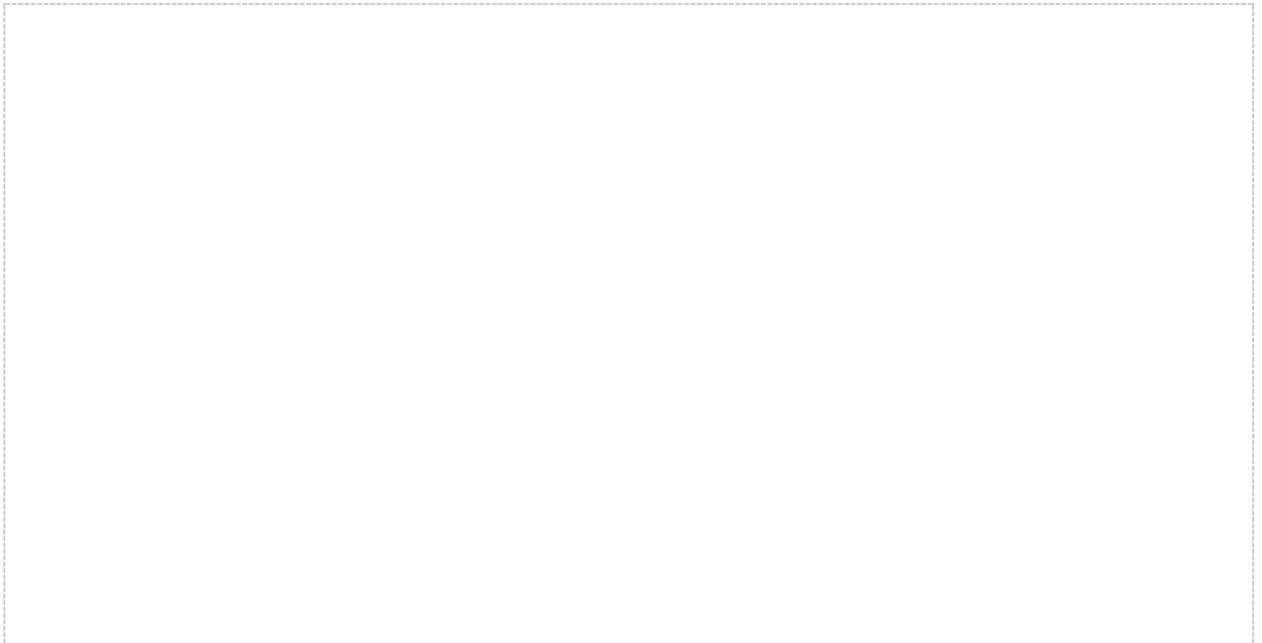
Nguồn diagram: Views/BuyerDashboardView.xaml



Hình 3.3. Giao diện Buyer Dashboard và khu vực cấu hình build

Vùng hình dành cho ảnh chụp danh sách request, chuyển trạng thái và khu vực thực hiện kiểm tra QC. Vị trí chèn thủ công được đánh dấu tại [Hình 3.4](#).

Nguồn diagram: Views/SellerDashboardView.xaml



Hình 3.4. Giao diện Seller Dashboard và xử lý yêu cầu lắp ráp

Vùng hình dành cho ảnh chụp KPI, quản lý tài khoản, Seller, catalog, đơn đăng ký và audit log. Vị trí chèn thủ công được đánh dấu tại [Hình 3.5](#).

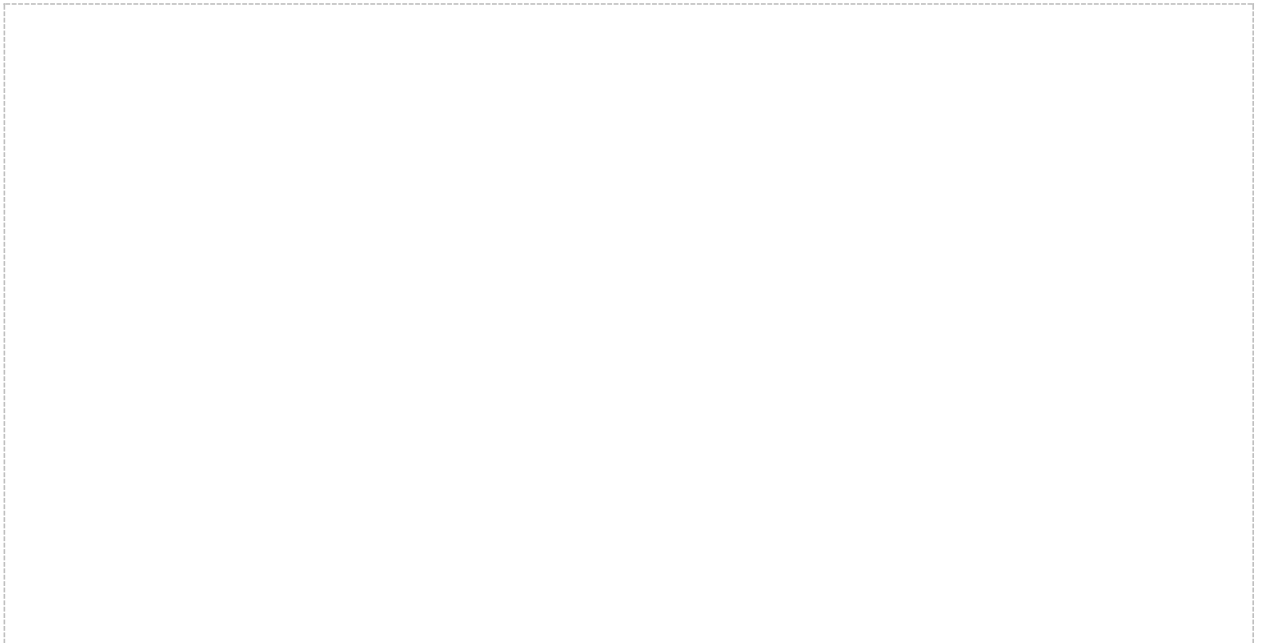
Nguồn diagram: Views/AdminDashboardView.xaml



Hình 3.5. Giao diện Admin Dashboard và quản trị hệ thống

Vùng hình dành cho ảnh chụp danh sách hội thoại và nội dung chat Buyer-Seller hoặc Admin-Seller. Vị trí chèn thủ công được đánh dấu tại [Hình 3.6](#).

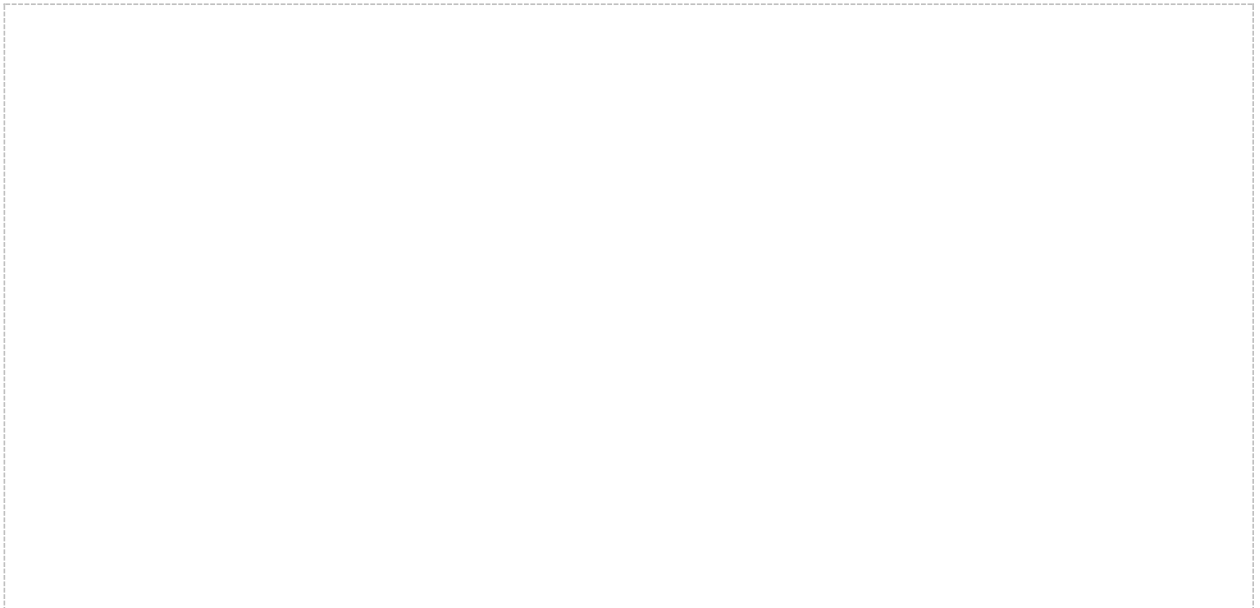
Nguồn diagram: Views/ChatView.xaml



Hình 3.6. Giao diện trao đổi tin nhắn theo vai trò

Vùng hình dành cho ảnh chụp thông tin tài khoản, lựa chọn ngôn ngữ và thao tác đăng xuất. Vị trí chèn thủ công được đánh dấu tại [Hình 3.7](#).

Nguồn diagram: *Views/UserMenuView.xaml*



Hình 3.7. Giao diện hồ sơ và menu người dùng

Mã nguồn 3.24 - Khu vực cấu hình build bằng XAML

```
<TextBlock Text="{loc:Tr Buyer_KitLabel}"
    Style="{StaticResource FieldLabel}"/>
<ComboBox ItemsSource="{Binding Kits}"
    SelectedItem="{Binding SelectedKit}"
    DisplayMemberPath="KitName"/>

<TextBlock Text="{loc:Tr Buyer_SwitchLabel}"
    Style="{StaticResource FieldLabel}"/>
<ComboBox ItemsSource="{Binding CompatibleSwitches}"
    SelectedItem="{Binding SelectedSwitch}"
    DisplayMemberPath="SwitchName"/>

<TextBlock Text="{Binding TotalPreview,
    StringFormat={}{0:N2} USD}"/>
<Button Content="{loc:Tr Buyer_SaveBuild}"
    Command="{Binding SaveBuildCommand}"/>
<Button Content="{loc:Tr Buyer_SendRequest}"
    Command="{Binding SendRequestCommand}"/>
```

Mã nguồn 3.25 - Lưu build và gửi request trong ViewModel

```
private async Task SaveBuildAsync()
{
    var build = CreateBuildFromCurrentSelection();
    var saved = await _buildService.SaveBuildAsync(build);
    _editingBuildId = saved.BuildId;
    await RefreshBuildsAsync();
    StatusMessage = TrFormat(
        "Buyer_BuildSaved",
        saved.Name,
        saved.TotalCostSnapshot);
}

private async Task SendRequestAsync()
{
    if (SelectedBuild is null)
        throw new InvalidOperationException(
            Tr("Buyer_SelectSavedBuildFirst"));
    if (SelectedSeller is null)
        throw new InvalidOperationException(
            Tr("Buyer_SelectSellerFirst"));

    await _requestService.SendRequestAsync(
        SelectedBuild.BuildId,
        CurrentUser.UserId,
        SelectedSeller.UserId,
        RequestNote);
    await RefreshRequestsAsync();
}
```

3.3. Kiểm thử và xác minh

Việc xác minh được thực hiện theo phạm vi không làm thay đổi dữ liệu runtime. Kết quả hiện tại được trình bày tại [Bảng 3.3](#); các tuyên bố kiểm thử lịch sử không được dùng thay cho runner còn tồn tại trong working tree.

Bảng 3.3. Kết quả build và xác minh tại thời điểm lập báo cáo

Mã	Nội dung	Kết quả	Ghi chú
T01	dotnet build	Pass	0 warning, 0 error; Custom_keyboard.dll được tạo cho net10.0-windows.
T02	VerifyPkToId_Source.ps1	Pass	21 PK id, 33 FK, 21 rename, version/startup guard và repository scan đạt.
T03	Database runtime postflight	Chưa đạt	Runtime tại thời điểm audit vẫn dùng 21 PK legacy; migration chưa chạy.
T04	VerifyRefactor.sql trên clean-seed	Chưa chạy trong lượt lập báo cáo	Script có kiểm tra schema/data nhưng có thể ghi dữ liệu fixture; không dùng trên runtime legacy.
T05	C# integration/UI automation	Không khả dụng	Phase6Verification và WpfUiVerification không còn trong working tree hiện tại.

3.4. Đánh giá mức độ hoàn thành

Mức độ hoàn thành theo từng mục tiêu được tổng hợp tại [Bảng 3.4](#).

Bảng 3.4. Đánh giá mức độ hoàn thành theo mục tiêu

Mục tiêu	Trạng thái	Đánh giá
Phân tích thiết kế	Đạt	Có ERD, Use Case, Sequence, Activity và mapping artifact.
Giao diện và nghiệp vụ ba vai trò	Đạt ở mức mã nguồn	View/ViewModel/Service/Repository cho Buyer, Seller, Admin đã được triển khai.
Schema SQL Server	Đạt ở source	DBML/schema/migration/verifier đã đồng bộ 21 PK id và 33 FK.
Vận hành trên runtime database	Chưa đạt	Cần backup, restore rehearsal, migration, postflight và smoke test.
QC phần cứng	Ngoài phạm vi	QC hiện dùng DeviceSimulator; không có ESP32/phần cứng thật.
Chat realtime SignalR	Chưa triển khai	Chat đã lưu DB; SignalR là phần mở rộng tùy chọn.

3.5. Kết luận chương

Chương 3 đã trình bày 21 DDL, bảy vị trí chèn ảnh giao diện và bốn đoạn mã và kết quả xác minh. Nghiệm thu vận hành còn chờ migration runtime và smoke test.

KẾT LUẬN

1. Mức độ hoàn thành nhiệm vụ

Các nhiệm vụ phân tích, thiết kế và triển khai mã nguồn cốt lõi đã được hoàn thành phần lớn: ứng dụng WPF theo MVVM, phân quyền Buyer/Seller/Admin, cấu hình build kit-based, request state machine, Seller application, chat lưu database, audit, analytics và QC mô phỏng đều có thành phần triển khai tương ứng.

Tuy nhiên, nhiệm vụ chưa thể được coi là hoàn thành ở mức vận hành cuối cùng. Runtime database được audit ngày 15/07/2026 vẫn dùng schema PK legacy, do đó cần hoàn tất backup, restore rehearsal, migration, postflight và smoke test trước khi nghiệm thu.

2. Mức độ phù hợp với mục đích đề tài

Thiết kế hiện tại phù hợp với mục đích số hóa quy trình cấu hình và đặt lắp ráp bàn phím tùy chỉnh. Mô hình kit-based giảm độ phức tạp so với việc chọn case/PCB/plate rời; snapshot pricing, validation và phân quyền hỗ trợ tính đúng đắn của dữ liệu; SQL Server tạo nền tảng lưu trữ tập trung và truy vết.

3. Hạn chế còn tồn tại

- Runtime database chưa áp dụng migration PK-to-id và chưa có bằng chứng smoke test sau migration.
- Phase6Verification và WpfUiVerification không còn trong working tree nên chưa thể chạy lại bộ C# integration/UI automation lịch sử.
- QC mới là mô phỏng phần mềm, chưa kết nối ESP32 hoặc trạm đo phần cứng thật.
- SignalR chat realtime chưa được triển khai; MQTT chủ yếu phục vụ thông báo request/status và telemetry tùy chọn.
- Năm cột logical enum gồm role_name và switch technology chưa có CHECK constraint đầy đủ trong schema hiện hành.

4. Hướng phát triển

- Diễn tập migration trên bản restore, chạy postflight, khôi phục runner integration/UI và ghi nhận biên bản smoke test.
- Bổ sung CHECK constraint, kiểm thử bảo mật/đầu vào/log, đồng thời hoàn thiện SignalR và cơ chế cancellation/versioning.
- Kết nối trạm QC phần cứng hoặc ESP32 khi phạm vi đề tài được mở rộng sang IoT.